

Manual de instalación Sistema de GEstión de Flota de Vehículos Eléctricos (SIGEFVE)

Maestría en Sistemas Computacionales

Materia:

Tecnologías de Programación

Catedrático:

Dr. Ulises Juárez Martínez

Alumnos:

Aldo Iván López Vivanco

Jessica Hernández Mayahua

Coré Husáí Martínez Belmonte

Tabla de contenido

Tabla de contenido	2
Introducción	4
Requisitos previos	5
Instalación en Windows	5
Instalar Docker Desktop	5
Descargar Docker Desktop	5
Requisitos del sistema	5
Habilitar WSL 2	5
Instalar Docker Desktop	5
Verificar la instalación	5
Instalar Git	6
Descargar el proyecto SIGEFVE	6
Usando Git	6
Descargando el proyecto	6
Instalación en macOS	6
Instalar Docker Desktop	6
Descargar Docker Desktop	6
Requisitos del sistema	6
Instalar Docker Desktop	6
Verifica la instalación	7
Instalar Git	7
Instalar Git	7
Descargar el proyecto SIGEFVE	7
Usando Git	7
Descargando el proyecto	7
Instalación en Linux	8
Instalar Docker Engine y Docker Compose	8
Ubuntu / Debian	8
Fedora / CentOS / RHEL	8
Arch Linux	9
Instalar Git (si no está instalado)	9
Ubuntu / Debian	9
Fedora /CentOS /RHEL	9
Arch Linux	9
Descargar el Proyecto SIGEFVE	9
Usando Git	9
Descargando el proyecto	9

Configuración del Proyecto	10
Crear el archivo de configuración .env	10
Crear archivo de variables de entorno	10
Configurar variables de entorno	10
Verificar la estructura del proyecto	11
Iniciar el Sistema	11
Inicia Docker	11
Construcción e inicialización de los contenedores	11
Espera a que todos los servicios inicien	12
Verifica que los contenedores estén ejecutándose	12
Verificación de la Instalación	12
Verifica que todos los servicios estén activos	12
Verifica las bases de datos	13
Solución de Problemas	13
Problema: "Docker no está ejecutándose"	13
Problema: "Puerto ya en uso"	13
Problema: "Permisos denegados" (Linux)	14
Problema: Los contenedores se detienen inmediatamente	14
Problema: "No se puede conectar a la base de datos"	14
Problema: Cambios en el código no se reflejan	14
Reiniciar completamente el sistema	14
Comandos útiles de Docker	15
Comandos para gestionar el sistema	16
Detener el sistema	16
Iniciar el sistema (después de detenerlo)	16
Reiniciar el sistema	16
Detener y eliminar todos los contenedores	16
Ver el uso de recursos	16
Notas Adicionales	16

Introducción

El Sistema de Gestión de Flota de Vehículos Eléctricos (SIGEFVE) es una solución integral diseñada bajo una arquitectura de microservicios para la administración, monitoreo y optimización de una flota de reparto urbano compuesta por vans, bicicletas y motos eléctricas.

El sistema permite la gestión en tiempo real de la telemetría (nivel de batería, ubicación, temperatura), la asignación eficiente de rutas y la detección proactiva de incidencias mecánicas o críticas mediante análisis de datos.

El sistema se compone de tres microservicios independientes e interconectados mediante API REST, cada uno desarrollado en el lenguaje más adecuado para su responsabilidad específica:

Microservicio de API Gateway (Go)

- Responsabilidad: Actúa como el único punto de entrada al sistema, gestionando la seguridad y el enrutamiento.
- Funciones clave: Autenticación y autorización mediante JWT, rate limiting por IP, registro de logs de peticiones y balanceo de carga hacia los servicios internos.
- Tecnologías: Go, Framework Gin.

Microservicio de gestión de flota (Java)

- Responsabilidad: Núcleo operativo del sistema. Maneja la persistencia transaccional de los vehículos y la ingesta de telemetría en alta frecuencia.
- Funciones clave: CRUD de vehículos, registro de telemetría (cada 15s), gestión de estados operativos (Disponible, En Ruta, etc.) y asignación de rutas de entrega.
- Tecnologías: Java, PostgreSQL, JDBC.

Microservicio de análisis y alertas (Python)

- Responsabilidad: Procesamiento analítico y detección de eventos.
- Funciones clave: Recepción asíncrona de datos, cálculo de estadísticas de rendimiento (eficiencia de batería, km recorridos) y generación de alertas automáticas (urgentes y preventivas).
- Tecnologías: Python, Flask, SQLite.

Requisitos previos

Antes de comenzar, asegúrate de tener al menos 8 GB de RAM y 10 GB de espacio libre en disco. El sistema utiliza Docker para ejecutar todos los microservicios, por lo que necesitarás instalar Docker Desktop o Docker Engine según tu sistema operativo.

Instalación en Windows

Instalar Docker Desktop

Descargar Docker Desktop

- Ve a la página oficial: <https://www.docker.com/products/docker-desktop>
- Haz clic en "Download for Windows".
- Descarga el instalador Docker Desktop Installer.exe.

Requisitos del sistema

- Windows 10 64-bit: Pro, Enterprise o Education (Build 19041 o superior).
- Windows 11 64-bit.
- Habilitar la característica WSL 2 (Windows Subsystem for Linux).

Habilitar WSL 2

- Este paso es opcional si es que ya está habilitado.
- Abre PowerShell como Administrador.
- Ejecuta los siguientes comandos:
`wsl --install`
`wsl --set-default-version 2`
- Reinicia tu computadora si es necesario.

Instalar Docker Desktop

- Ejecuta el archivo Docker Desktop Installer.exe.
- Sigue el asistente de instalación.
- Asegúrate de marcar la opción "Use WSL 2 instead of Hyper-V".
- Haz clic en "Ok" y espera a que termine la instalación.
- Reinicia tu computadora cuando se solicite.

Verificar la instalación

- Abre PowerShell, CMD o la aplicación de Terminal.
- Ejecuta:
`docker --version`
`docker-compose --version`
- Deberías ver las versiones instaladas.

Instalar Git

Este paso es opcional, si lo que deseas es clonar el repositorio desde GitHub, de lo contrario, puedes ir al paso 3.

- Descarga Git desde: <https://git-scm.com/download/win>
- Ejecuta el instalador y sigue las opciones por defecto
- Verifica la instalación:
`git --version`

Descargar el proyecto SIGEFVE

Usando Git

- Abre PowerShell, CMD o la aplicación de Terminal.
- Ejecuta:
`git clone https://github.com/aldovnlv/sigefve.git`
`cd sigefve-main`

Descargando el proyecto

- Abre tu navegador web de preferencia.
- En la barra de búsqueda escribe:
<https://codeload.github.com/aldovnlv/sigefve/zip/refs/heads/main>
- Lo anterior descargará un archivo ZIP, que debe extraerse en una carpeta de tu elección.
- Navega a la carpeta de tu elección desde la Terminal.

Instalación en macOS

Instalar Docker Desktop

Descargar Docker Desktop

- Ve a: <https://www.docker.com/products/docker-desktop>
- Haz clic en "Download for Mac".
- Selecciona la versión apropiada:
 - Mac con chip Apple (M1/M2/M3): Descarga "Apple Chip".
 - Mac con chip Intel: Descarga "Intel Chip".

Requisitos del sistema

- macOS 11 o superior.
- Al menos 4 GB de RAM.

Instalar Docker Desktop

- Abre el archivo Docker.dmg descargado.
- Arrastra el icono de Docker a la carpeta Applications.
- Abre Docker desde Applications.

- Sigue las instrucciones en pantalla.
- Docker puede solicitar tu contraseña para instalar componentes del sistema.

Verifica la instalación

- Abre Terminal.
- Ejecuta:
`docker --version`
`docker-compose --version`
- Deberías ver las versiones instaladas.

Instalar Git

Este paso es opcional, si lo que deseas es clonar el repositorio desde GitHub, de lo contrario, puedes ir al paso 3.

Instalar Git

- Usualmente ya viene preinstalado en sistemas macOS, así que solo hay que comprobar que esto es correcto ejecutando el siguiente comando:
`git --version`
- Si no está instalado, entonces:
 - Descarga desde: <https://git-scm.com/download/mac>
 - O instala Xcode Command Line Tools:
`xcode-select --install`

Descargar el proyecto SIGEFVE

Usando Git

- Abre Terminal y ejecuta:
`git clone https://github.com/aldovnlv/sigefve.git`
`cd sigefve-main`

Descargando el proyecto

- Abre tu navegador web de preferencia.
- En la barra de búsqueda escribe:
<https://code.load.github.com/aldovnlv/sigefve/zip/refs/heads/main>
- Lo anterior descargará un archivo ZIP, que debe extraerse en una carpeta de tu elección.
- Navega a la carpeta de tu elección desde la Terminal.

Instalación en Linux

Instalar Docker Engine y Docker Compose

Las instrucciones varían según la distribución. A continuación se muestran los pasos para las distribuciones más comunes.

Ubuntu / Debian

- Actualiza el sistema ejecutando los comandos:
`sudo apt update`
`sudo apt upgrade -y`
- Instalar dependencias:
`sudo apt install -y apt-transport-https ca-certificates curl software-properties-common`
- Agrega la clave GPG de Docker:
`curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg`
- Agrega el repositorio de Docker:
`echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null`
- Instala Docker:
`sudo apt update`
`sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin`
- Agrega el usuario actual al grupo docker:
`sudo usermod -aG docker $USER`
`newgrp docker`
- Verifica la instalación:
`docker --version`
`docker compose version`

Fedora / CentOS / RHEL

- Instala dependencias:
`sudo dnf -y install dnf-plugins-core`
- Agrega el repositorio de Docker:
`sudo dnf config-manager --add-repo`
<https://download.docker.com/linux/fedora/docker-ce.repo>
- Instala Docker:
`sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin`
- Inicia Docker:

- ```
sudo systemctl start docker
sudo systemctl enable docker
```
- Agrega tu usuario al grupo docker  

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```
  - Verifica la instalación  

```
docker --version
```

```
docker compose version
```

### **Arch Linux**

- Instala Docker:  

```
sudo pacman -S docker docker-compose
```
- Inicia Docker:  

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```
- Agrega tu usuario al grupo docker  

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

## **Instalar Git (si no está instalado)**

### **Ubuntu / Debian**

```
sudo apt install -y git
```

### **Fedora /CentOS /RHEL**

```
sudo dnf install -y git
```

### **Arch Linux**

```
sudo pacman -S git
```

## **Descargar el Proyecto SIGEFVE**

### **Usando Git**

- En la terminal, ejecuta:  

```
git clone https://github.com/aldoynlv/sigefve.git
```

```
cd sigefve-main
```

### **Descargando el proyecto**

En la terminal, ejecuta:

```
wget https://code.load.github.com/aldoynlv/sigefve/zip/refs/heads/main
```

```
unzip sigefve-main.zip
```

```
cd sigefve-main
```

---

## Configuración del Proyecto

Una vez que tengas Docker instalado y el proyecto descargado, sigue estos pasos:

### Crear el archivo de configuración **.env**

En la carpeta raíz del proyecto, copia el archivo de ejemplo:

#### Windows (PowerShell):

```
Copy-Item .env.example .env
```

#### macOS / Linux:

```
cp .env.example .env
```

### Crear archivo de variables de entorno

Abre el archivo **.env** con tu editor de texto favorito

#### Windows:

```
notepad .env
```

#### macOS:

```
nano .env
```

#### Linux:

```
nano .env
```

También puedes usar vim, gedit, kate, etc.

### Configurar variables de entorno

Ajusta las variables según tus necesidades:

```
Ruta base del proyecto (usa . para la carpeta actual)
SIGEFVE_BIND_BASE_PATH=.
```

```
Ruta donde se guardarán los datos persistentes
SIGEFVE_PERSISTENCE_PATH=./data
```

```
Puerto para el frontend web
SIGEFVE_WEB_PORT=8080
```

```
Puerto para la API de Java
```

```
JAVA_PORT=8585
```

```
Puerto para la API de Go
SIGEFVE_GO_PORT=8090
```

**Importante para Windows:** Si decides usar una ruta absoluta en `SIGEFVE_BIND_BASE_PATH`, usa barras diagonales `/` en lugar de barras invertidas `\`:

```
Correcto:
SIGEFVE_BIND_BASE_PATH=C:/Users/TuUsuario/Proyectos/sigefve-main
```

```
Incorrecto:
SIGEFVE_BIND_BASE_PATH=C:\Users\TuUsuario\Proyectos\sigefve-main
```

## Verificar la estructura del proyecto

Asegúrate de que tu proyecto tenga la siguiente estructura:

```
sigefve-main/
├── docker-compose.yml
├── .env.example
├── .env (recién creado)
├── web-app/
├── python/
├── java/
├── go/
├── frontend/
└── data/ (se creará automáticamente)
```

---

## Iniciar el Sistema

### Inicia Docker

Asegúrate de que Docker Desktop esté ejecutándose:

- **Windows/macOS:** Abre Docker Desktop desde el menú de aplicaciones
- **Linux:** Docker debería estar ejecutándose como servicio. Verifica con:  

```
sudo systemctl status docker
```

### Construcción e inicialización de los contenedores

Abre tu terminal en la carpeta raíz del proyecto y ejecuta:

**Windows (PowerShell o CMD):**

```
docker-compose up -d
```

**macOS / Linux:**

```
docker compose up -d
```

El parámetro **-d** ejecuta los contenedores en segundo plano (modo detached).

**Espera a que todos los servicios inicien**

La primera vez tomará varios minutos porque Docker debe:

- Descargar las imágenes base (Debian, PostgreSQL, Nginx)
- Instalar dependencias en cada contenedor
- Compilar el código
- Inicializar las bases de datos

Puedes ver el progreso con:

```
docker-compose logs -f
```

Presiona **Ctrl+C** para salir de los logs.

**Verifica que los contenedores estén ejecutándose**

```
docker-compose ps
```

Deberías ver algo como:

| NAME             | STATUS       | PORTS                   |
|------------------|--------------|-------------------------|
| frontend-react   | Up 2 minutes |                         |
| nginx-apigo      | Up 2 minutes | 0.0.0.0:13000->80/tcp   |
| postgresql_go    | Up 2 minutes | 0.0.0.0:54402->5432/tcp |
| postgresql_java  | Up 2 minutes | 0.0.0.0:54302->5432/tcp |
| sigefve_frontend | Up 2 minutes | 0.0.0.0:8080->80/tcp    |
| sigefve_go       | Up 2 minutes | 0.0.0.0:8090->8080/tcp  |
| sigefve_java     | Up 2 minutes | 0.0.0.0:8585->8585/tcp  |
| sigefve-python   | Up 2 minutes | 0.0.0.0:8822->5001/tcp  |

**Verificación de la Instalación****Verifica que todos los servicios estén activos**

Abre tu navegador web y accede a las siguientes URLs:

1. **Frontend React (Nginx)**
  - URL: <http://localhost:13000>
  - Deberías ver la interfaz principal del sistema.

2. **Frontend Web (Apache):**
  - URL: `http://localhost:8080`
  - Deberías ver la aplicación web alternativa.
3. **API de Python:**
  - URL: `http://localhost:8822/apidocs`
  - Deberías ver la documentación Swagger.
4. **API de Java:**
  - URL: `http://localhost:8585`
  - Deberías ver la documentación de la API.
5. **API de Go:**
  - URL: `http://localhost:8090/health`
  - Deberías ver un mensaje JSON indicando que todo está arriba.

## Verifica las bases de datos

Puedes conectarte a las bases de datos PostgreSQL usando cualquier cliente como pgAdmin, DBeaver o desde la línea de comandos:

### PostgreSQL para Java:

```
docker exec -it postgresql_java psql -U postgres -d sigefve
```

### PostgreSQL para Go:

```
docker exec -it postgresql_go psql -U postgres -d api-go
```

Para salir de psql, escribe `\q` y presiona Enter.

---

## Solución de Problemas

### Problema: "Docker no está ejecutándose"

#### Solución:

- **Windows/macOS:** Abre Docker Desktop y espera a que inicie completamente
- **Linux:** Ejecuta `sudo systemctl start docker`

### Problema: "Puerto ya en uso"

Error: `Bind for 0.0.0.0:8080 failed: port is already allocated`

#### Solución:

1. Identifica qué está usando el puerto:
  - **Windows:** `netstat -ano | findstr :8080`
  - **macOS/Linux:** `lsof -i :8080`

2. Detén el proceso o cambia el puerto en tu archivo `.env`
3. Reinicia los contenedores: `docker-compose down && docker-compose up -d`

### Problema: "Permisos denegados" (Linux)

**Error:** `Got permission denied while trying to connect to the Docker daemon socket`

**Solución:**

```
sudo usermod -aG docker $USER
newgrp docker
```

Cierra sesión y vuelve a iniciar sesión, o reinicia tu computadora.

### Problema: Los contenedores se detienen inmediatamente

**Solución:**

Revisa los logs del contenedor específico:

```
docker-compose logs [nombre-del-servicio]
```

1. Los problemas comunes incluyen:
  - Errores de sintaxis en el código
  - Dependencias faltantes
  - Problemas de conectividad con la base de datos

### Problema: "No se puede conectar a la base de datos"

**Solución:**

- Verifica que los contenedores de PostgreSQL estén ejecutándose:  
`docker-compose ps postgresql_java postgresql_go`
- Espera unos segundos más, las bases de datos tardan en inicializar
- Revisa los logs:  
`docker-compose logs postgresql_javadocker-compose logs postgresql_go`

### Problema: Cambios en el código no se reflejan

**Solución:**

Detén los contenedores:

```
docker-compose down
```

Reconstruye las imágenes:

```
docker-compose up -d --build
```

### Reiniciar completamente el sistema

Si necesitas empezar desde cero:

# Detener y eliminar todos los contenedores

```
docker-compose down
```

# Eliminar volúmenes (esto borrará los datos de las bases de datos)

```
docker-compose down -v
```

# Eliminar la carpeta de datos persistentes

```
rm -rf data/ # macOS/Linux
```

```
rmdir /s data # Windows
```

# Iniciar de nuevo

```
docker-compose up -d
```

### **Comandos útiles de Docker**

# Ver todos los contenedores

```
docker-compose ps
```

# Ver logs de todos los servicios

```
docker-compose logs
```

# Ver logs de un servicio específico

```
docker-compose logs sigefve_java
```

# Seguir logs en tiempo real

```
docker-compose logs -f
```

# Reiniciar un servicio específico

```
docker-compose restart sigefve_java
```

# Detener todos los servicios

```
docker-compose stop
```

# Iniciar todos los servicios

```
docker-compose start
```

# Detener y eliminar todos los contenedores

```
docker-compose down
```

# Acceder a la terminal de un contenedor

```
docker exec -it sigefve_java /bin/bash
```

---

## Comandos para gestionar el sistema

### Detener el sistema

```
docker-compose stop
```

### Iniciar el sistema (después de detenerlo)

```
docker-compose start
```

### Reiniciar el sistema

```
docker-compose restart
```

### Detener y eliminar todos los contenedores

```
docker-compose down
```

### Ver el uso de recursos

```
docker stats
```

---

## Notas Adicionales

- **Primer inicio:** La primera vez que ejecutes `docker-compose up -d` tomará más tiempo porque Docker debe descargar todas las imágenes base y compilar los microservicios.
- **Datos persistentes:** Los datos de las bases de datos y los módulos de Node.js se guardan en la carpeta especificada en `SIGEFVE_PERSISTENCE_PATH` (por defecto `./data`). Esto significa que tus datos no se perderán al reiniciar los contenedores.
- **Actualizaciones del código:** Si modificas el código fuente, necesitarás reiniciar el contenedor correspondiente para que los cambios tomen efecto. Para cambios en el código de Java o Go, puede ser necesario reconstruir con `docker-compose up -d --build`.
- **Firewall:** Si no puedes acceder a los servicios desde tu navegador, verifica que tu firewall no esté bloqueando los puertos. Agrega excepciones para los puertos 8080, 8585, 8090, 8822 y 13000.



- **Recursos del sistema:** Asegúrate de que Docker Desktop tenga suficientes recursos asignados. En Windows/macOS puedes ajustar esto en la configuración de Docker Desktop (Settings → Resources).
- 

**¡Instalación completada!** Tu aplicación SIGEFVE ahora debería estar funcionando correctamente.